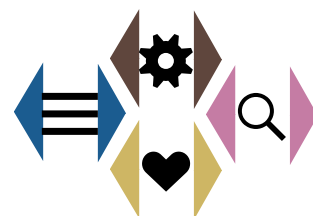


Performance Engineering

FOR PARALLEL APPLICATIONS



pahole: Analysing Memory Layout of Complex Data Structures With Ease

SUNDAY, 5TH NOVEMBER 2023: Putting together this blog post feels like a positive stride! As I mentioned in the previous post, ([core-to-core latency tool](#)), I'm aiming to integrate more consistent writing into my routine. While it took a month to pen this down, it's progress from the previous year 😊. Hoping that the upward trend will continue...!

During the past few weekends, I've been reading about the capabilities of the [perf C2C \(Cache To Cache\)](#) tool. As I went through the documentation and examples, a tool called [pahole](#) caught my attention. I'm a bit surprised I haven't stumbled upon it earlier. With scientific code optimizations, we carefully examine the memory layout of data structures for cache efficiency. While, as developers, we typically take into consideration padding and alignment requirements when designing data structures, I've never utilized tools like [pahole](#) for inspecting the memory layout of C/C++ data structures. While working on a large codebase or code developed by someone else, I thought this tool might be handy and hence thought it would be a good candidate for a short blog post!

Background

I don't need to delve into details of [why and how of padding & alignment](#). Data structures can encompass data members of various types and sizes, each subject to different alignment requirements. To make sure each data member satisfies these memory alignment requirements, compilers need to insert additional space or "holes". As a developer, we also want to look into data structure layout for several reasons:

- Based on the target architecture's alignment requirements, compilers automatically add padding to make the data members aligned to the word size of the CPU. This can increase the total size of the object and hence we want to be aware of the overhead.
- Developers might want to explicitly add padding to achieve specific alignment or memory layout. For example, many CPU architectures fetch data in cache-sized blocks and hence align data structures to reduce the chance of crossing cache lines can improve cache utilization.

- Without proper padding, independent variables that share the same cache line can lead to false sharing. This causes unnecessary cache coherence and can result in a significant performance penalty.

When developing data structures with these considerations in mind, an important aspect is to understand the memory layout of these structures. If you're dealing with your own codebase, you likely possess a good understanding of the existing choices. However, what if you encounter new code or complex data structures where manual inspection isn't a feasible option? How do you systematically explore and understand the memory layouts of these structures? This is precisely where `pahole` becomes invaluable!

Pahole In Action

`pahole` (Poke-a-hole) tool is developed as part of the [dwarves](#) suite of utilities. When we compile applications with debugging information (e.g. `-g` compiler flag), compilers include DWARF information in object or binary. This information includes details about types, variables, and their locations in memory. `pahole` reads this DWARF information and presents it in an intuitive and easily understandable form. This can help developers understand the memory layout, alignment, and padding of data structures and narrow down "holes" inserted by compilers.

The best way to understand its usefulness is by looking at the examples. So let's dive in!

Install

On Linux distributions like Ubuntu, we can use a standard system package manager to install `dwarves`:

```
sudo apt-get update
sudo apt-get install dwarves -y
```

Installing `pahole` from source is not difficult as well:

```
# install dependencies
apt-get install libdw-dev libdwarf-dev libelf-dev zlib1g-dev -y

# get source repo and following typical cmake install
git clone https://git.kernel.org/pub/scm/devel/pahole/pahole.git
cd pahole/
git submodule update --init --recursive
mkdir build && cd build

cmake -DBUILD_SHARED_LIBS=OFF -DCMAKE_INSTALL_PREFIX=$HOME/install/pahole ..
make -j && make install
```

```
export PATH=$HOME/install/pahole/bin:$PATH
```

With this, we have pahole command available. Here are some of the CLI options that I found interesting or relevant to this blog post:

```
$ pahole --help
Usage: pahole [OPTION...] FILE

-B, --bit_holes=NR_HOLES  Show only structs at least NR_HOLES bit holes
-c, --cacheline_size=SIZE  set cacheline size to SIZE
-C, --class_name=CLASS_NAME  Show just this class
-E, --expand_types        expand class members
    --flat_arrays          Flat arrays
    --hex                  Print offsets and sizes in hexadecimal
-H, --holes=NR_HOLES      show only structs with at least NR_HOLES holes
-i, --contains=CLASS_NAME Show classes that contains CLASS_NAME
-I, --show_decl_info       Show the file and line number where the tags were
                           defined
    --lang=LANGUAGES       Only consider compilation units written in these
                           languages
    --lang_exclude=LANGUAGES Don't consider compilation units written in
                           these languages
-M, --show_only_data_members show only the members that use space in the
                           class layout
-y, --prefix_filter=PREFIX include PREFIXed classes
-r, --rel_offset          show relative offsets of members in inner structs
-R, --reorganize          reorg struct trying to kill holes
-s, --sizes               show size of classes
-S, --show_reorg_steps     show the struct layout at each reorganization step
-z, --hole_size_ge=HOLE_SIZE show only structs with at least one hole
-?, --help                Give this help list
```

Examples

To demonstrate the use of the pahole, let's consider below data structure:

```
1 |
2 | // Structure representing an ion channel.
3 | struct Channel {
4 |     char name[4];           // Name of the ion channel.
5 |     double conductance;     // Conductance of the ion channel.
6 |     int id;                 // Unique ID of the ion channel.
7 |     double rpotential;      // Reversal potential of the ion channel.
8 |     float current;          // Current associated with the ion channel.
9 |     double prop[4];         // Additional properties array.
10 | };
11 |
12 | // Structure representing a compartment of a neuron.
13 | struct Compartment {
14 |     Channel sodium_channel; // Sodium ion channel characteristics.
15 |     Channel potassium_channel; // Potassium ion channel characteristics.
16 |     int location;           // Location or segment of the neuron compartment.
17 | };
18 |
19 | // Structure representing a neuron.
20 | struct Neuron {
21 |     int gid;                // Unique ID for a neuron cell.
22 |     Compartment compartments[3]; // Array of neuron compartments.
23 | };
24 |
25 | // Allocate 5 objects to represent neuron cells.
26 | Neuron cells[5];
```

27 |

Feel free to disregard specific terminology used in the above structures. I'm using the terminology of [ion channels](#), [compartments](#), and [neurons](#), both for a change and to have some meaningful types (from the computational neuroscience domain where I currently work).

The first step is to compile our example `test.cpp` with debugging information using the flag `"-g"`:

```
$ g++ test.cpp -g -c
```

This will produce an object, `test.o`, which can be used with `pahole`. Here are specific questions that can effectively demonstrate the utility of `pahole`.

What is the arrangement of structs in memory, and what is the size and location of paddings?

The first obvious task is to understand the memory layout of the data structure. This can be easily accomplished by invoking the `pahole` command with an object file. `pahole` adds C-style comments to explain the memory layout. Following each member, two numbers are provided within C-style comments, indicating the offset and size of the data member in bytes. If the compiler has introduced "holes," `pahole` mentions their sizes. At the end, a summary of sizes and holes is provided.

```
$ pahole --class_name=Channel test.o
struct Channel {
    char                name[4];                /*    0    4 */
    /* XXX 4 bytes hole, try to pack */
    double              conductance;            /*    8    8 */
    int                 id;                     /*   16    4 */
    /* XXX 4 bytes hole, try to pack */
    double              rpotential;             /*   24    8 */
    float               current;                /*   32    4 */
    /* XXX 4 bytes hole, try to pack */
    double              prop[4];                /*   40   32 */
    /* size: 72, cachelines: 2, members: 6 */
    /* sum members: 60, holes: 3, sum holes: 12 */
    /* last cacheline: 8 bytes */
};
```

In the above example, we used the `--class_name` CLI option to only analyze the `Channel` type (for brevity). The output report is not difficult to understand:

- name member starts at offset 0 and has a size of 4 bytes.
- conductance starts at offset 8 and has a size of 8 bytes. It is preceded by a 4-byte hole due to the 8-byte alignment requirement of the double type.
- id starts at offset 16 and has a size of 4 bytes.
- rpotential, starting at offset 24 with an 8-byte size, is also preceded by a 4-byte hole to adhere to the memory alignment requirements for the double type.
- current member starts at offset 32 and occupies 4 bytes.
- prop, preceded by a 4-byte hole, starts at offset 40 with a size of 32 bytes.

Once all members are listed, the data structure summary is provided:

- The total size of the struct (including padding) is 72 bytes and spans over 2 cachelines (assuming a default cacheline size of 64 bytes)
- The combined size of all members is 60 bytes. There are a total of 3 holes and their total size is 12 bytes.
- This type uses only 8 bytes of the second cacheline (denoted here as the "last cacheline")

Just as a bonus example, you can take a look at the output when we compile the test for a 32-bit target architecture. As the alignment requirement for a double variable is typically 4 bytes, we can see how the data structure is laid out:

```
$ g++ -g test.cpp -c -m32
$ pahole --class_name=Channel test.o
struct Channel {
    char                name[4];           /*    0    4 */
    double              conductance;       /*    4    8 */
    int                 id;                /*   12    4 */
    double              rpotential;        /*   16    8 */
    float               current;           /*   24    4 */
    double              prop[4];           /*   28   32 */

    /* size: 60, cachelines: 1, members: 6 */
    /* last cacheline: 60 bytes */
} __attribute__((__packed__));
```

How can data members be rearranged for optimal memory layout?

In the preceding output, we observed that the Channel type extends over two cachelines, totaling 72 bytes, despite the actual size of its members being 60 bytes. What if we instruct pahole to rearrange the members for a more compact structure?

```
$ pahole --show_reorg_steps --reorganize --class_name=Channel test.o
/* Moving 'id' from after 'conductance' to after 'name' */
struct Channel {
    char                name[4];           /*    0    4 */
```

```

    int            id;                /*      4      4 */
    double         conductance;       /*      8      8 */
    double         rpotential;        /*     16      8 */
    float          current;           /*     24      4 */

    /* XXX 4 bytes hole, try to pack */

    double         prop[4];           /*     32     32 */

    /* size: 64, cachelines: 1, members: 6 */
    /* sum members: 60, holes: 1, sum holes: 4 */
}

/* Final reorganized struct: */
struct Channel {
    char           name[4];           /*      0      4 */
    int            id;                /*      4      4 */
    double         conductance;       /*      8      8 */
    double         rpotential;        /*     16      8 */
    float          current;           /*     24      4 */

    /* XXX 4 bytes hole, try to pack */

    double         prop[4];           /*     32     32 */

    /* size: 64, cachelines: 1, members: 6 */
    /* sum members: 60, holes: 1, sum holes: 4 */
}; /* saved 8 bytes and 1 cacheline! */

```

If we read the comments, it's evident what pahole has done. By shifting 'id' from its original position after 'conductance' to follow 'name,' it avoided two holes of 8 bytes. As a result, the Channel type can now fit neatly onto a single cacheline. While this is a trivial example, in more complex codebases, I believe pahole can be quite practical to quickly experiment with data structure reorganizations.

How does the data structure will be laid out with a cacheline size of X bytes?

Nowadays most widely-used Intel/AMD/ARM CPUs have 64-byte cachelines, making this question less interesting. But CPUs with different cacheline sizes are not foreign to us (e.g. Power9 with a cacheline size of 128-byte). pahole provides a CLI option, `--cacheline_size`, allowing for the exploration of potential memory layouts with an arbitrary cacheline size:

```

$ pahole --cacheline_size=128 test.o
struct Channel {
    char           name[4];           /*      0      4 */

    /* XXX 4 bytes hole, try to pack */

    double         conductance;       /*      8      8 */
    int            id;                /*     16      4 */

    /* XXX 4 bytes hole, try to pack */

    double         rpotential;        /*     24      8 */
    float          current;           /*     32      4 */

    /* XXX 4 bytes hole, try to pack */

```

```

double                prop[4];                /*    40    32 */

/* size: 72, cachelines: 1, members: 6 */
/* sum members: 60, holes: 3, sum holes: 12 */
/* last cacheline: 72 bytes */
};

```

Understanding nested data structures and their organization in memory layout

Until now we focused on a single type Channel for brevity. Let's now try to look at the Neuron type that encompasses the Compartment and Channel types. We will use --expand_types CLI option to show us a detailed memory layout for the Neuron type:

```

$ pahole --expand_types --class_name=Neuron test.o
struct Neuron {
    int                gid;                /*    0

/* XXX 4 bytes hole, try to pack */

    struct Compartment {
        struct Channel {
            char        name[4];          /*    8

/* XXX 4 bytes hole, try to pack */

            double      conductance;      /*   16
            int         id;               /*   24

/* XXX 4 bytes hole, try to pack */

            double      rpotential;       /*   32
            float       current;         /*   40

/* XXX 4 bytes hole, try to pack */

            double      prop[4];          /*   48
        } sodium_channel; /*    8    72 */
/* --- cacheline 1 boundary (64 bytes) was 16 bytes ago --- */
        struct Channel {
            char        name[4];          /*   80

/* XXX 4 bytes hole, try to pack */

            double      conductance;      /*   88
            int         id;               /*   96

/* XXX 4 bytes hole, try to pack */

            double      rpotential;       /*  104
            float       current;         /*  112

/* XXX 4 bytes hole, try to pack */

            double      prop[4];          /*  120
        } potassium_channel; /*   80    72 */
/* --- cacheline 2 boundary (128 bytes) was 24 bytes ago --- */
        int            location;         /*  152
    } compartments[3]; /*    8    456 */

```

```
/* size: 464, cachelines: 8, members: 2 */  
/* sum members: 460, holes: 1, sum holes: 4 */  
/* last cacheline: 16 bytes */  
};
```

Isn't that useful?

That's all for today! This is what I have gathered so far, and I believe it provides sufficient background information for you to dive into the details if you find this tool relevant to your needs. You can read more about this tool in the [2007 paper in the Proceedings of the Linux Symposium](#). The author has provided some other examples and use cases that you might find interesting.

By the way, when using pahole with shared libraries from a large production codebase, be sure to explore the CLI options for filtering data structures of particular interest. Alternatively, consider using pahole with individual object files. In my own project experimentation, naive execution of `pahole libfoo.so` ended up with 49k lines output! 😊

Credit

I'm not very familiar with the developer community of performance tools outside HPC, but discovering and learning new tools like pahole is always exciting! (even though [git log](#) says it was born 17 years ago!). This post is mostly for my self-learning using examples/documentation put together by the community. So, full credit and gratitude to [Arnaldo Carvalho de Melo](#) for developing and his ongoing efforts! 🥂



NOVEMBER 3, 2023



PRAMOD KUMBHAR



© *About Performance Measurement, Analysis And Optimization At Scale!*